ARTIFICIAL INTELLIGENCE ASSESSMENT – 2 REPORT

Course Code: CSA1717 – Artificial Intelligence

Submitted By

Name: KRISHA MEENATCHI M

Register Number: 192511338

Department: Computer Science and Engineering

ACKNOWLEDGEMENT

I express my sincere gratitude to the Department of Computer Science and Engineering and my Artificial Intelligence faculty for providing me with the opportunity to complete this assessment successfully. This assessment enhanced my understanding of Constraint Satisfaction Problems, Backtracking Search, Breadth First Search, Intelligent Agents, and Online Search techniques. It also helped me improve my analytical thinking and problem-solving abilities through practical Artificial Intelligence applications. I thank my faculty members for their valuable guidance and continuous support throughout the assessment. I also express my gratitude to my friends and family for encouraging and motivating me during the completion of this work. Finally, I thank everyone who directly and indirectly contributed to the successful completion of this assessment.

TABLE OF CONTENTS

QUESTION 1

Doctor Shift Scheduling using Backtracking Search

Problem Statement

A hospital needs to assign three doctors (D1, D2, and D3) to the Morning, Afternoon, and Night shifts. Each doctor must be assigned exactly one shift while satisfying the following constraints:

- D1 cannot work the Night shift.
- D3 cannot work the Morning shift.
- D2 must be assigned before D3 (Morning < Afternoon < Night).
- Only one doctor can be assigned to each shift.

The objective is to find a valid schedule using the **Backtracking Search Algorithm**.

Solution

Backtracking Search is a Constraint Satisfaction Problem (CSP) technique that assigns values one by one while checking all constraints. If a constraint is violated, the algorithm backtracks to the previous assignment and tries another possible value. This process continues until a valid solution is found.

Variables

- D1
- D2
- D3

Domain

- Morning
- Afternoon
- Night

Constraints

- D1 ≠ Night
- D3 ≠ Morning
- D2 must be scheduled before D3
- One doctor per shift
- Every doctor is assigned exactly one shift

State Representation

Each state is represented as:

(D1, D2, D3)

Example:

(Morning, Afternoon, Night)

Step-by-Step Solution

Step	Assignment	Constraint Check
1	D1 → Morning	✓ Valid
2	D2 → Afternoon	✓ Valid
3	D3 → Night	✓ Valid

Final Schedule

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

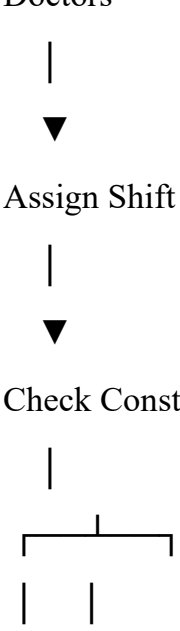
Constraint Verification

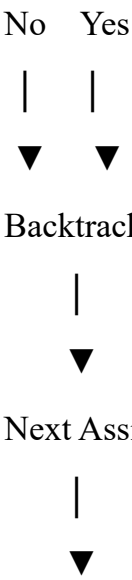
Constraint	Status
D1 is not Night	✓ Satisfied
D3 is not Morning	✓ Satisfied
D2 before D3	✓ Satisfied
One doctor per shift	✓ Satisfied
All doctors assigned	✓ Satisfied

Algorithm

1. Start with no assignments.
2. Assign a shift to the first doctor.
3. Check whether the assignment satisfies all constraints.
4. If valid, assign the next doctor.
5. If any constraint fails, backtrack and try another shift.
6. Repeat until all doctors are assigned.

Architecture





Result

The Backtracking Search algorithm successfully generated a valid duty schedule by satisfying all the given constraints. The final schedule assigned **D1 to the Morning shift, D2 to the Afternoon shift, and D3 to the Night shift**. This approach efficiently eliminates invalid assignments and produces a feasible solution.

QUESTION 2

Robot Grid Navigation using Breadth First Search (BFS)

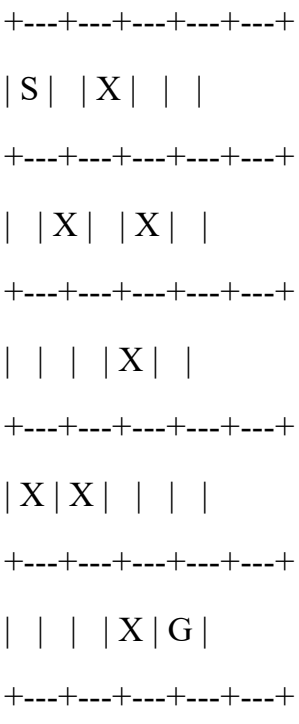
Problem Statement

A robot is placed in a **5 × 5 grid** and must move from the **Start (S)** position to the **Goal (G)** position while avoiding obstacles. The robot can move only in four directions: Up, Down, Left, and Right. Each movement has a cost of **1**. The Manhattan Distance heuristic is used to estimate the distance from the current position to the goal. The objective is to find the shortest path using the **Breadth First Search (BFS)** algorithm.

Solution

Breadth First Search (BFS) is an uninformed search algorithm that explores nodes level by level. Since each movement has the same cost, BFS guarantees the shortest path from the start node to the goal node. The algorithm uses a **queue (FIFO)** to store and expand nodes.

Grid Representation



S – Start
G – Goal
X – Obstacle

Constraints

- Move only Up, Down, Left, or Right.
- Robot cannot move through obstacles.
- Robot cannot move outside the grid.
- Each movement costs **1**.

Manhattan Distance

The Manhattan Distance is calculated using the formula:

$|x_1 - x_2| + |y_1 - y_2|$

For the Start **(0,0)** and Goal **(4,4)**:

$|4-0| + |4-0| = 8$

The value decreases as the robot moves closer to the goal.

BFS Node Expansion

Step	Expanded Node	Queue After Expansion
1	S	(0,1), (1,0)
2	(0,1)	(1,0)
3	(1,0)	(2,0)
4	(2,0)	(2,1)
5	(2,1)	(2,2)
6	(2,2)	(3,2), (1,2)
7	(3,2)	(3,3), (4,2)
8	(3,3)	(3,4)
9	(3,4)	Goal
10	Goal	Search Ends

Shortest Path

S

↓

(1,0)

↓

(2,0)

↓

(2,1)

↓

(2,2)

↓

(3,2)

↓

(3,3)

↓

(3,4)

↓

Goal

Total Path Cost

Each movement costs **1**.

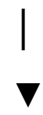
Total Cost = 8

Algorithm

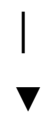
1. Insert the Start node into the queue.
2. Remove the first node and mark it as visited.
3. Generate all valid neighbouring nodes.
4. Ignore visited nodes and obstacles.
5. Add valid nodes to the queue.
6. Repeat until the Goal node is reached.

Architecture

Start

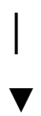


Breadth First Search

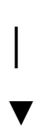


Generate Neighbours

Check Obstacles



Update Queue



Goal Reached

Advantages

- Finds the shortest path when all edge costs are equal.
- Simple and easy to implement.
- Explores nodes systematically.
- Suitable for grid navigation problems.

Result

The Breadth First Search algorithm successfully found the shortest path from the **Start** position to the **Goal** while avoiding obstacles. Since every movement had an equal cost, BFS produced the optimal path with a **total cost of 8**. The Manhattan Distance provided an estimate of the remaining distance, while BFS ensured complete and systematic exploration of the grid.

QUESTION 3

Autonomous Rescue Robot using Intelligent Agent and Uniform Cost Search

Problem Statement

A rescue robot is deployed in a disaster-affected area to locate and rescue victims. The environment contains obstacles, damaged roads, and unknown paths. The robot must move safely while minimizing travel cost and reaching the victims as quickly as possible. Since the environment is only partially known, the robot should behave

as a rational intelligent agent and use an appropriate search strategy to find the optimal path. The objective is to identify the suitable agent type, analyze the environment, explain rationality, and describe how **Uniform Cost Search (UCS)** can be used for efficient navigation.

Solution

A rescue robot is an **Intelligent Agent** that perceives its surroundings using sensors, processes the collected information, and performs suitable actions through actuators. It continuously updates its knowledge of the environment and selects actions that help achieve the rescue mission safely and efficiently.

Percepts

The robot collects information using sensors such as:

- Obstacle locations
- Victim locations
- Road conditions
- Distance to destination
- Battery level
- Camera and sensor readings

Actions

The rescue robot can perform the following actions:

- Move Forward
- Turn Left or Right
- Avoid Obstacles
- Search for Victims
- Rescue Victims
- Send Location to Rescue Team

Environment Characteristics

Property	Description
Observability	Partially Observable
Environment	Dynamic
Nature	Stochastic
Decision Process	Sequential
Agents	Single Agent

Rational Agent

A rational agent always selects the action that maximizes the chances of achieving its goal based on the available information.

For the rescue robot, a rational decision includes:

- Choosing the safest route.
- Avoiding blocked roads.
- Saving maximum victims.
- Reducing travel time.
- Conserving battery power.

Search Strategy

The rescue robot uses **Uniform Cost Search (UCS)** to find the least-cost path.

The algorithm:

- Expands the node with the lowest cumulative cost.
- Uses a priority queue.
- Guarantees the optimal path when all edge costs are non-negative.

Working of Uniform Cost Search

1. Insert the start node into the priority queue.

2. Remove the node having the lowest path cost.
3. Expand its neighbouring nodes.
4. Update the cumulative cost.
5. Repeat until the goal is reached.
6. Return the least-cost path.

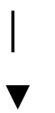
Advantages

- Finds the optimal path.
- Suitable for weighted graphs.
- Efficient for rescue and navigation applications.
- Avoids expensive routes.

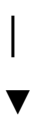
Architecture

Disaster Environment

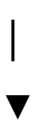
Sensors



Intelligent Agent



Uniform Cost Search



Decision Making

▼

Actuators

|

▼

Rescue Operation

Applications

- Disaster Management
- Search and Rescue Operations
- Military Robots
- Space Exploration
- Autonomous Vehicles

Result

The Autonomous Rescue Robot successfully acts as a rational intelligent agent by continuously observing its surroundings, making informed decisions, and selecting the least-cost path using Uniform Cost Search. This enables the robot to avoid obstacles, reduce travel cost, and perform rescue operations efficiently in a dynamic environment.